

MediaPipe Gesture-AprilTag Distance Sorting

1. Function Description

After the program starts, the camera captures images and recognizes gestures. The gestures are 1 to 5. By recognizing the gestures, the distance threshold is calculated. The robotic arm will change its pose to detect AprilTags in the image and calculate their height and distance. If any is less than the distance threshold, the robotic arm will lower the gripper to grip it and place it at the set position, then return to the AprilTag detection pose to continue recognition.

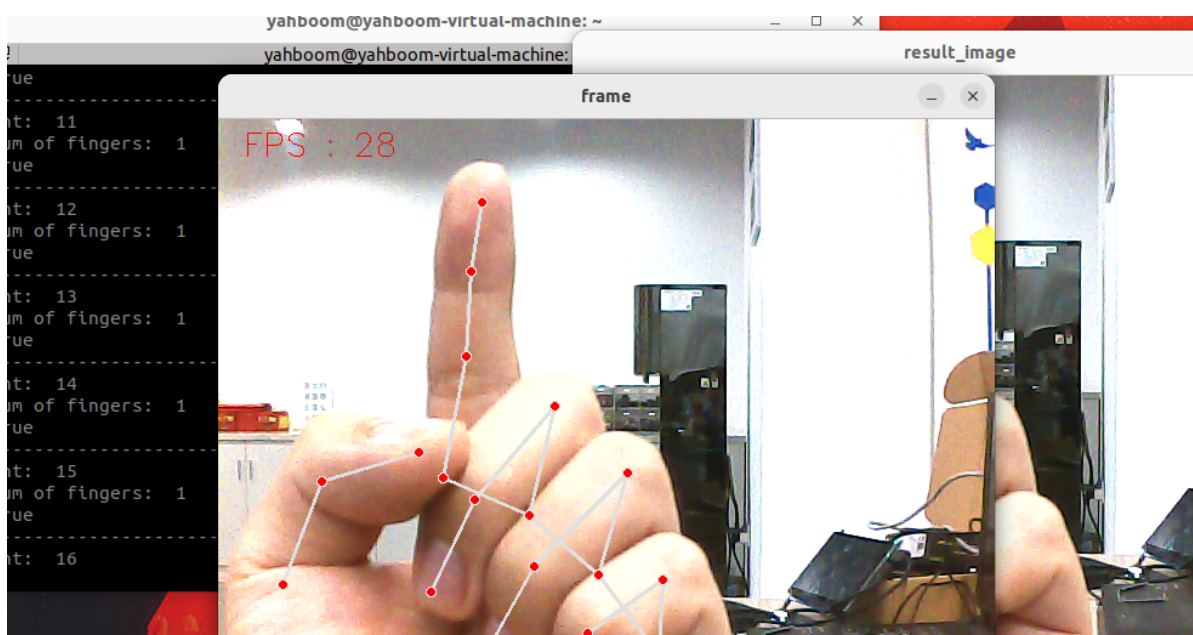
2.1 Startup Commands

```
# Start camera and inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start mediapipe gesture recognition
ros2 run dofbot_sorting_3d mediapipe_gesture
# Distance detection
ros2 run dofbot_sorting_3d apriltag_gesture_dist
# Start robotic arm gripping
ros2 run dofbot_sorting_3d grasp
```

2.2 Operation

After the program starts, the robotic arm will initially present the gesture recognition pose. The recognizable gestures are 1 to 5. Gesture recognition takes about 3 seconds to wait, then the robotic arm changes pose to the AprilTag detection and recognition pose. Press the spacebar to start recognition. If any detected AprilTag distance is less than the calculated distance threshold, the robotic arm will lower the gripper to grip that AprilTag block and place it at the set position. After placement is complete, the robotic arm returns to the AprilTag recognition pose, and the next recognition requires pressing the spacebar again. After that, the robotic arm returns to the gesture recognition pose.

Distance threshold calculation = preset distance + gesture recognition result * 0.02



3. Core Code Analysis

3.1 mediapipe_gesture.py

Code path: The main code has been explained in the previous tutorial, so I won't repeat it here

```
/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/mediapipe_gesture.py
```

3.2 apriltag_list_Dist.py

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/apriltag_gesture_dist.py
```

Import necessary libraries,

```
import rclpy
from rclpy.node import Node
import cv2
import numpy as np
from sensor_msgs.msg import Image

from std_msgs.msg import Float32, Bool
from cv_bridge import CvBridge
import cv2 as cv
from dt_apriltags import Detector
import threading
from dofbot_sorting_3d.vutils import draw_tags
from dofbot_interface.srv import Kinemarics
from dofbot_interface.msg import *
from std_msgs.msg import Float32, Bool, Int16
import time
from Arm_Lib import Arm_Device
from dofbot_sorting_3d.compute_joint5 import *
import os
```

Initialize program parameters, create publishers and subscribers,

```
def __init__(self):
    # Initialize the ROS2 node with the name 'apriltag_detect'
    # InitializeROS2 node, node name is 'apriltag_detect'
    super().__init__('apriltag_detect')

    # Create an instance of the robotic arm device class
    # Create robotic arm device class instance
    self.Arm = Arm_Device()

    # Define initial joint angles for the robot arm
    # Define robotic arm initial joint angles
    self.init_joints = [90.0, 120.0, 0.0, 0.0, 90.0, 0.0]

    # Variable to store the value of joint(Int16 type)
    # Variable for storing joint value (Int16 type)
```

```

self.joint5 = Int16()

# Flag indicating whether a target has been detected
# Flag indicating whether target is detected
self.detect_flag = False

# Flag indicating whether height calculation is needed
# Flag indicating whether height calculation is needed
self.compute_height = True

# Index of the detected AprilTag
# Detected AprilTag index
self.index = None

# Create a service client for kinematic calculations
# Create kinematic calculation client
self.client = self.create_client(Kinemarics, 'dofbot_kinemarics')

# Publisher for publishing AprilTag position information
# Publisher for AprilTag position information
self.pos_info_pub = self.create_publisher(
    AprilTagInfo, "PosInfo", qos_profile=10)

# Subscriber for receiving grasp completion status
# Subscriber for grasp completion status
self.sub_grasp_done = self.create_subscription(
    Bool, 'grasp_done', self.GraspStatusCallback, qos_profile=1)

# Subscriber for receiving gesture ID commands
# Subscriber for gesture ID commands
self.sub_gestureID = self.create_subscription(
    Int16, "GestureId", self.GestureIDCallback, 1)

# Publisher for sending reset gesture command
# Publisher for sending reset gesture command
self.pub_reset = self.create_publisher(
    Bool, "/reset_gesture", qos_profile=10)

# Bridge for converting between OpenCV images and ROS2 Image messages
# Bridge for converting between OpenCV images and ROS2 Image messages
self.rgb_bridge = CvBridge()

# Flag indicating whether position publishing is active
# Flag indicating whether position publishing is active
self.pubPos_flag = False

# Previous time record for timing operations
# Previous time record for timing operations
self.pr_time = time.time()

# Initialize the AprilTag detector with specific parameters
# Initialize AprilTag detector with specific parameters
self.at_detector = Detector(searchpath=['apriltags'],
                             families='tag36h11',
                             nthreads=8,
                             quad_decimate=2.0,
                             quad_sigma=0.0,
                             refine_edges=1,

```

```

        decode_sharpening=0.25,
        debug=0)

# Distance parameter
# Distance parameter
self.dist = 0.13

# Subscriber for receiving raw camera images
# Subscriber for raw camera images
self.subscription = self.create_subscription(
    Image, '/image_raw', self.ImageCallback, 1)

# Target distance for detection (in meters)
# Detection target distance
self.target_dist = 0.14

# Detection flag
# Detection flag
self.detect = False

# Publisher for setting joint angle
# Publisher for setting joint angle
self.pub_joint5 = self.create_publisher(Int16, "set_joint5", 10)

```

Let's look at the main image callback function ImageCallback,

```

def ImageCallback(self, color_frame):
    # Convert ROS2 Image message to OpenCV RGB image format
    # Convert ROS2 Image message to OpenCV RGB image format
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')

    # Create a copy of the RGB image for drawing results
    # Create RGB image copy for drawing results
    result_image = np.copy(rgb_image)

    # Detect AprilTags in the grayscale version of the image
    # Detect AprilTags in grayscale version of image
    # Convert RGB to Grayscale for detection
    # Convert RGB to grayscale for detection
    tags = self.at_detector.detect(cv2.cvtColor(
        rgb_image, cv2.COLOR_RGB2GRAY), False, None, 0.025)

    # Sort detected tags by their ID in ascending order
    # Sort detected tags by ID in ascending order
    # Tags may already be sorted, but explicit sorting ensures order
    # Tags may be sorted, explicit sorting ensures order
    tags = sorted(tags, key=lambda tag: tag.tag_id)

    # Draw bounding boxes and centers on the result image
    # Draw bounding boxes and centers on result image
    # Corners in red, center in green
    # Corners red, center green
    draw_tags(result_image, tags, corners_color=(
        0, 0, 255), center_color=(0, 255, 0))

    # Check for spacebar key press
    # Check for spacebar press

```

```

# Pressing space enables position publishing
# Press space to enable position publishing
key = cv2.waitKey(10)
if key == 32:
    self.pubPos_flag = True

# Process each detected tag
# Process each detected tag
if len(tags) > 0:
    for i in range(len(tags)):
        # Only process if position publishing is enabled
        # Only process when position publishing is enabled
        if self.pubPos_flag == True:
            # Get the center coordinates of the tag
            # Get tag center coordinates
            center_x, center_y = tags[i].center
            cx = center_x
            cy = center_y
            cz = self.dist

            # Calculate normalized coordinates (a, b)
            # Calculate normalized coordinates (a, b)
            # a: horizontal offset from image center, normalized
            # a: horizontal offset from image center, normalized
            # b: vertical offset from image center, normalized with scaling
            factor

            # b: vertical offset from image center, with scaling factor
            (a, b) = (round(((cx - 320) / 4000), 5), round(((480 - cy) /
3000) * 0.8+0.13, 5))
            print("a: ", a)
            print("b: ", b)

            # Calculate edge vector for joint computation
            # Calculate edge vector for joint calculation
            # Difference between first two corners of the tag
            # Difference between first two corners of tag
            vx = int(tags[i].corners[0][0]) - \
                int(tags[i].corners[1][0])
            vy = int(tags[i].corners[0][1]) - \
                int(tags[i].corners[1][1])

            # Compute the required angle for joint based on edge orientation
            # Compute required angle for joint based on edge orientation
            target_joint5 = compute_joint5(vx, vy)
            print("target_joint5: ", target_joint5)

            # If position publishing is enabled and target distance is
            reached

            # If position publishing enabled and target distance reached
            if self.pubPos_flag == True and b <= self.target_dist:
                # Disable further position publishing
                # Disable further position publishing
                self.pubPos_flag = False

                # Create and publish joint command
                # Create and publish joint command
                joint5 = Int16()
                joint5.data = int(target_joint5)

```

```

        self.pub_joint5.publish(joint5)

        # Create and publish AprilTag position information
        # Create and publish AprilTag position information
        pos = AprilTagInfo()
        pos.x = float(b) # Forward/backward distance
        pos.y = float(-a) # Left/right offset
        pos.z = 0.03
        pos.id = tags[i].tag_id
        self.pos_info_pub.publish(pos)

    # If no valid tag was found at target distance
    # If no valid tag found at target distance
    if self.detect_flag == False and self.pubPos_flag == True:
        # Send reset command to gesture system
        # Send reset command to gesture system
        reset = Bool()
        reset.data = True
        self.pub_reset.publish(reset)

        # Reset position publishing flag
        # Reset position publishing flag
        self.pubPos_flag = False

        # Reset target distance to default value
        # Reset target distance to default value

        self.target_dist = 0.14
        print("Do not find the target dist tag.")

    # Convert result image back to BGR format for display
    # Convert result image back to BGR format for display
    result_image = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)

    # Calculate and display FPS
    # Calculate and display FPS
    cur_time = time.time()
    fps = str(int(1/(cur_time - self.pr_time)))
    self.pr_time = cur_time
    cv2.putText(result_image, fps, (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    # Display the result image in a window
    # Display result image in window
    cv2.imshow("result_image", result_image)
    key = cv2.waitKey(1)

```

Gesture detection callback function GestureIDCallback

```

def GestureIDCallback(self, msg):
    # Update the target distance based on received gesture ID
    # Update target distance based on received gesture ID
    # Formula: new_target_dist = gesture_value * 0.01 * 2 + current_target_dist
    # Formula: new_target_dist = gesture_value * 0.01 * 2 + current_target_dist
    # This allows dynamic adjustment of approach distance via gestures
    # This allows dynamic adjustment of approach distance via gestures
    self.target_dist = msg.data * 0.01 * 2 + self.target_dist

```

```
# Print the updated target distance for debugging
# Print updated target distance for debugging
print("self.target_dist: ", self.target_dist)

# Wait for 3 seconds to allow system stabilization
# Wait 3 seconds for system stabilization
time.sleep(3.0)

# Enable position publishing after the delay
# Enable position publishing after delay
self.pubPos_flag = True
```

3.3 grasp.py

The previous tutorial has explained this, so I won't repeat it here.